Programming Assignment Report

Design and Analysis of Algorithms

Name : Gyana Ranjan Pattjoshi

Registration Number :2411021060941

Problem Name: Knights on Chessboard

Problem Statement:

KnightL is a chess piece that moves in an L shape. We define the possible moves of KnightL(a, b) as any movement from some position

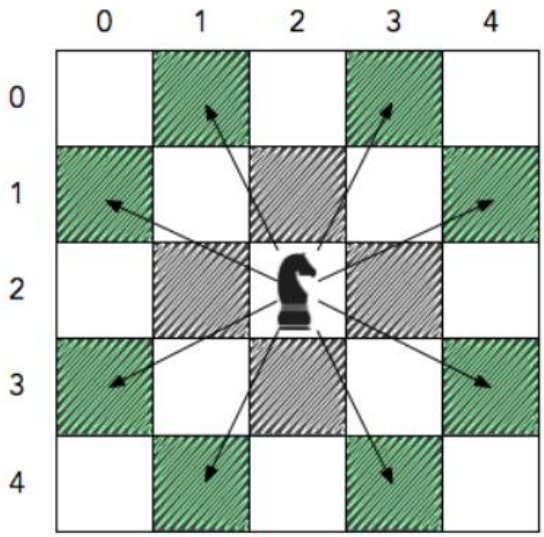
(x1;y1) to some (a2; y2) satisfying either of the following:

x2=x1±a and y2=y1 ± b, or

x2=x1±b and y2 =y1 ± a

Note that (a, b) and (b, a) allow for the same exact set of movements. For example, the diagram below depicts the possible locations that

KnightL(1,2) or KnightL(2, 1) can move to from its current location at the centre of a 5 x 5 chessboard:



Problem HackerRank Link:

https://www.hackerrank.com/challenges/knightl-on-chessboard/problem

GitHub Submission Link:

https://github.com/gyanranjanpattjoshi/DAA-Lab/blob/main/DAA/KnightMoves.java

Solution Steps:

In this Hacker Rank KnightL on a Chessboard problem solution we have given the value of n

for an n x n chessboard, answer the following question for each (a,b) pair where 1 <= a,b < n:

What is the minimum number of moves it takes for KnightL(a,b) to get from position (0,0) to

position (n - 1, n - 1)? If it's not possible for the Knight to reach that destination, the answer

is -1 instead. then print the answer for each KnightL(a,b) according to the Output Format specified below. Here below written the solution steps -:

1. **Start the program.**
2. **Read the input value n, which represents the size of the chessboard $n \times n$.**
3. **Create a 2D array result[n][n] to store the minimum number of moves required for each knight movement (a, b).**
4. **Use two nested loops to iterate through all possible knight move combinations where $1 \leq a < n$ and $1 \leq b < n$.**

5. **For each pair (a, b), check whether the result for (b, a)has already been computed.**
   If yes, reuse that value to avoid redundant calculations.
6. **Initialize a visited matrix visited[n][n] to keep track of visited positions on the chessboard.**
7. **Create a queue to implement the Breadth First Search (BFS) traversal.**
8. **Insert the starting position (0,0) into the queue with the number of moves initialized to 0.**
9. **Mark the starting position visited[0][0] = true.**
10. **While the queue is not empty, remove the front element from the queue.**
11. **From the current position, generate all eight possible knight moves using the step sizes (a, b).**
12. **For each generated position:**

- **Check whether the position lies within the chessboard boundaries.**
- **Check whether the position has not been visited.**

13. **If the new position is valid:**

- **Mark it as visited.**
- **Add it to the queue with the move count increased by 1.**

14. **If the position reaches the destination (n−1, n−1):**

- **Record the number of moves required.**
- **Store the value in the result matrix.**
- **Stop BFS for that movement pattern.**

15. **If BFS completes without reaching the destination, store -1 in the result matrix.**
16. **Print the result matrix showing the minimum moves required for each (a, b)knight movement.**
17. **End the program.**

Complete JAVA Code:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    static class Point {
        int x;
        int y;
        int turn;
        Point(int x,  int y, int turn) {
            this.x = x;
            this.y = y;
            this.turn = turn;
        }

    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int n = in.nextInt();

        int[][] result = new int[n][n];
        for (int a = 1; a < n; a++) {
            for (int b = 1; b < n; b++) {
                if (result[b][a] != 0) {
```

```
    System.out.print(result[b][a]);
    if (b < n - 1)
        System.out.print(" ");
} else {
boolean[][] visited = new boolean[n][n];
Queue<Point> queue = new LinkedList<>();
queue.add(new Point(0, 0, 0));
visited[0][0] = true;

boolean hit = false;
while (!queue.isEmpty()) {
    Point pt = queue.poll();

    int x = pt.x - a;
    int y = pt.y - b;
    if (x >= 0 && y >= 0 && !visited[x][y]) {
        if (x == n - 1 && y == n - 1) {
            System.out.print(pt.turn + 1);
            hit = true;
            result[a][b] = pt.turn + 1;
            if (b < n - 1)
                System.out.print(" ");
            break;
        } else {
            visited[x][y] = true;
            queue.add(new Point(x, y, pt.turn + 1));
        }
    }
```

```
x = pt.x + a;
y = pt.y - b;
if (x < n && y >= 0 && !visited[x][y]) {
    if (x == n - 1 && y == n - 1) {
        System.out.print(pt.turn + 1);
        hit = true;
        result[a][b] = pt.turn + 1;
        if (b < n - 1)
            System.out.print(" ");
        break;
    } else {
        visited[x][y] = true;
        queue.add(new Point(x, y, pt.turn + 1));
    }
}

x = pt.x - a;
y = pt.y + b;
if (x >= 0 && y < n && !visited[x][y]) {
    if (x == n - 1 && y == n - 1) {
        System.out.print(pt.turn + 1);
        hit = true;
        result[a][b] = pt.turn + 1;
        if (b < n - 1)
            System.out.print(" ");
        break;
    } else {
        visited[x][y] = true;
        queue.add(new Point(x, y, pt.turn + 1));
```

```
    }
}

x = pt.x + a;
y = pt.y + b;
if (x < n && y < n && !visited[x][y]) {
    if (x == n - 1 && y == n - 1) {
        System.out.print(pt.turn + 1);
        hit = true;
        result[a][b] = pt.turn + 1;
        if (b < n - 1)
            System.out.print(" ");
        break;
    } else {
        visited[x][y] = true;
        queue.add(new Point(x, y, pt.turn + 1));
    }
}

x = pt.x - b;
y = pt.y - a;
if (x >= 0 && y >= 0 && !visited[x][y]) {
    if (x == n - 1 && y == n - 1) {
        System.out.print(pt.turn + 1);
        hit = true;
        result[a][b] = pt.turn + 1;
        if (b < n - 1)
            System.out.print(" ");
        break;
```

```
    } else {
        visited[x][y] = true;
        queue.add(new Point(x, y, pt.turn + 1));
    }
}

x = pt.x + b;
y = pt.y - a;
if (x < n && y >= 0 && !visited[x][y]) {
    if (x == n - 1 && y == n - 1) {
        System.out.print(pt.turn + 1);
        hit = true;
        result[a][b] = pt.turn + 1;
        if (b < n - 1)
            System.out.print(" ");
        break;
    } else {
        visited[x][y] = true;
        queue.add(new Point(x, y, pt.turn + 1));
    }
}

x = pt.x - b;
y = pt.y + a;
if (x >= 0 && y < n && !visited[x][y]) {
    if (x == n - 1 && y == n - 1) {
        System.out.print(pt.turn + 1);
        hit = true;
        result[a][b] = pt.turn + 1;
```

```
            if (b < n - 1)
                System.out.print(" ");
            break;
        } else {
            visited[x][y] = true;
            queue.add(new Point(x, y, pt.turn + 1));
        }
    }

    x = pt.x + b;
    y = pt.y + a;
    if (x < n && y < n && !visited[x][y]) {
        if (x == n - 1 && y == n - 1) {
            System.out.print(pt.turn + 1);
            hit = true;
            result[a][b] = pt.turn + 1;
            if (b < n - 1)
                System.out.print(" ");
            break;
        } else {
            visited[x][y] = true;
            queue.add(new Point(x, y, pt.turn + 1));
        }
    }
}
if (!hit) {
    result[a][b] = -1;
    System.out.print("-1");
    if (b < n - 1)
```

```
                System.out.print(" ");
            }
            }
        }
        if (a < n - 1) {
            System.out.println();
        }
    }
  }
}
```

Observation/Solution:

The given Java program calculates the minimum number of moves required for a knight to reach the bottom-right corner of an $n \times n$chessboard starting from position (0,0).

Unlike a standard chess knight which moves in fixed (2,1) steps, this program considers all possible knight movements (a,b) where $1 \leq a, b < n$. For every pair (a, b), the program determines whether the knight can reach the destination (n−1,n−1) and calculates the minimum number of moves required.

The program uses the Breadth First Search (BFS) algorithm to explore all reachable positions on the board. BFS is used because it guarantees the shortest path in an unweighted grid. A queue data structure is used to store the positions to be explored, while a visited matrix ensures that the same cell is not processed multiple times.

From each position, the knight can move in eight possible directions based on the step sizes $a$and b. During traversal, the program checks whether the new position is within the boundaries of the chessboard and whether it has already been visited.

If the knight reaches the destination cell $(n-1, n-1)$, the program records the number of moves taken. If the destination cannot be reached for a particular movement pattern, the program outputs -1.

An optimization is used in the program where results for (a,b) and (b,a) are reused since knight movements are symmetric. This avoids unnecessary recomputation.

Finally, the program prints a matrix of size (n−1) × (n−1) where each value represents the minimum number of moves required for the knight with movement pattern (a,b) to reach the destination.